POS verification evidence summary

Execution began 2026-09-23 and resumed 2026-09-24. This file consolidates the architecture/QA reviewer's evidence. It does not certify production release. Commands are run from the repository root. Relative evidence links refer to this directory.

Scope and provenance

- Read `AGENTS.md`, architecture-review.md, route-inventory.json, release-checks.md, recorded gate results, and the payment gate source/test changes.
- Preserved the existing dirty workspace and prior logs. This resumed reviewer changed no application code, schema, database, financial record, or governance approval. The payment gate correction was already present from the interrupted review and required no further source edit.
- Source inventory covers 13 page patterns / 26 localized targets: four direct operational surfaces (POS, terminals, drawer, cash history) and nine downstream evidence pages. Inventory is not browser pass evidence. Browser work is owned by the coordinating reviewer.
- Static checks below are source/report checks. They do not prove runtime permissions, financial atomicity, provider authenticity, or database concurrency.

Recorded checks

Check	Exact invocation	Recorded result	Evidence
Offline replay static gate	`node scripts/offline-pos-fiscal-replay-gate.js --mode fail --out what-next/pos-enterprise-modernization/2026-09-23/evidence/offline-gate.md --json-out what-next/pos-enterprise-modernization/2026-09-23/evidence/offline-gate.json`	Exit 0; 16/16 checks ready	offline-gate.json, static-gate-exits.json
Payment cash-truth static gate, corrected	`node scripts/payment-cash-truth-gate.js --mode fail --out what-next/pos-enterprise-modernization/2026-09-23/evidence/payment-gate.md --json-out what-next/pos-enterprise-modernization/2026-09-23/evidence/payment-gate.json`	Exit 0; 14/14 checks ready. Initial exit 1 retained in the original exits log	payment-gate.json, payment-gate-correction-exits.json
Inventory boundary static gate	`node scripts/inventory-boundary-gate.js --mode fail --out what-next/pos-enterprise-modernization/2026-09-23/evidence/inventory-gate.md --json-out what-next/pos-enterprise-modernization/2026-09-23/evidence/inventory-gate.json`	Exit 1; 20 active findings, 52 allowed findings	inventory-gate.json, static-gate-exits.json
Service boundary static gate	`node scripts/service-boundary-gate.js --mode fail --out what-next/pos-enterprise-modernization/2026-09-23/evidence/service-gate.md --json-out what-next/pos-enterprise-modernization/2026-09-23/evidence/service-gate.json`	Exit 0; 0 active / 17 allowed findings	service-gate.json, static-gate-exits.json

Check	Exact invocation	Recorded result	Evidence
Workflow assurance source gate	`node scripts/workflow-assurance-release-gate.js --mode fail --out what-next/pos-enterprise-modernization/2026-09-23/evidence/workflow-gate.md --json-out what-next/pos-enterprise-modernization/2026-09-23/evidence/workflow-gate.json`	Exit 0; 38/38 checks, 11/11 indexes, 2/2 engine checks ready	workflow-gate.json, static-gate-exits.json
Public receipt configuration, local mode	`node scripts/public-receipt-token-config-gate.js --mode fail --release auto --out what-next/pos-enterprise-modernization/2026-09-23/evidence/receipt-config-gate.md --json-out what-next/pos-enterprise-modernization/2026-09-23/evidence/receipt-config-gate.json`	Exit 0; 4/4 local checks. Production secret absent; release mode was false	receipt-config-gate.json, config-governance-exits.json
POS contract governance	`node scripts/pos-g1-contract-gate.js --mode fail --no-write`	Exit 1; 13/13 technical checks, 0/11 authentic decision approvals	pos-contract-gate.json, config-governance-exits.json
POS program governance	`node scripts/pos-enterprise-program-gate.js --mode fail --no-write`	Exit 1; 11/11 technical checks, 0/10 program gates, 0/9 external evidence items	pos-program-gate.json, config-governance-exits.json
Prisma schema validation	`npm run prisma:validate`	Exit 0 recorded 2026-09-23; schema valid. No migration applied	prisma-validate.log, prisma-validate-exit.json
Payment gate and test ESLint, resumed	`node node_modules/eslint/bin/eslint.js scripts/payment-cash-truth-gate.js scripts/__tests__/payment-cash-truth-gate.test.js`	Exit 0 recorded 2026-09-24; no diagnostics	payment-gate-eslint-resumed.log, payment-gate-eslint-resumed-exit.json
Localized POS access helper ESLint	`node node_modules/eslint/bin/eslint.js app/[locale]/(dashboard)/dashboard/pos/pos-route-data-access.ts app/[locale]/(dashboard)/dashboard/pos/pos-route-access.tsx`	Exit 0; no diagnostics	pos-route-locale-eslint.log, pos-route-locale-eslint-exit.json
Translation parity	`node what-next/pos-enterprise-modernization/2026-09-23/evidence/check-localization-parity.cjs`	Exit 0; four dictionaries have matching keys and named interpolation tokens	localization-parity.json
Project TypeScript check, resumed	`npm run typecheck`	Exit 0; no diagnostics. Started 2026-09-24 02:51:44 UTC, completed 03:04:07 UTC	typecheck-resumed.log, typecheck-resumed-exit.json

Check	Exact invocation	Recorded result	Evidence
Project TypeScript check after late source changes	`npm run typecheck`	Exit 2; one TS2339 in `hooks/useCashPaymentHistoryWorkbench.ts:116`: the inferred failure union does not guarantee `code`. The current property guard was written during that run; see provenance below	typecheck-final.log, typecheck-final-start.json, typecheck-final-exit.json
Project TypeScript check after current-source/config freeze	`npm run typecheck`	Exit 0; no diagnostics. 03:56:06–04:12:21 UTC; all seven recorded source/config hashes unchanged	typecheck-final-stable.log, start hashes, exit/end hashes, integrity comparison
Production application build	`NEXT_DIST_DIR=.next-pos-enterprise-build-20260924`, requested `npm run build:app -- --skip-clean`	Next child exit 1; wrapper exit 1; timeout after 30 minutes. Compiled in 24.7 minutes; type validation and packaging completion not proven	build-isolated-outcome.json, build-isolated.log, build-isolated-exit.json

The first resumed typecheck/lint attempts did not run because the sandbox denied opening their log paths. The scoped escalated retries then started normally; this was a filesystem restriction, not an automatic approval rejection. The terminal wrapper's initial empty/stale exit cannot be used as check evidence.

Payment gate correction and regression proof

The old static rule expected the literal `type: PaymentExceptionType.AMOUNT_MISMATCH`. Current reconciliation code classifies ambiguous references separately and persists `type: exceptionType`, so the old marker caused a false failure. The corrected check still requires all four amount/currency mismatch markers and verifies classification, exception persistence, the classified type, and a `continue` before automatic matching.

payment-gate-red-tests.log records the updated positive fixture failing against the old check: 1 failed, 14 passed. payment-gate-green-tests.log records 15 passing tests after correction; payment-gate-correction-exits.json records test exit 0. Negative cases cover each missing amount/currency check, missing classification/persistence/wiring/skip, and skip moved after auto-match. These are static-gate regressions; they do not replace reconciliation runtime tests. The saved log does not contain the original complete Jest command, so no reconstructed command is asserted as execution provenance.

Localization result and limits

localization-parity.json records:

Dictionary	English leaves	French leaves	Missing keys / interpolation mismatches
`pos`	407	407	0
`cashDrawerDashboard`	154	154	0

Dictionary	English leaves	French leaves	Missing keys / interpolation mismatches
`cashPaymentHistory`	126	126	0
Terminal component's static `copy`	87	87	0

The terminal dictionaries were parsed with the TypeScript syntax tree without executing the component. The check verifies leaf names and named interpolation-token parity; it does not establish translation quality, complete ICU syntax, or absence of hardcoded labels. The UI owner moved the terminal activity labels `Sessions` and `Sales` into English/French dictionaries; the resumed parity run now verifies 87 leaves in each language.

Release blockers preserved

1. **Inventory boundary remains failing.** Nineteen findings point to older `what-next/workflow-modernization/2026-09-05-run-01/lanes/environment/` fixture scripts; one points to `scripts/modern-modules-lifecycle-fixture.ts:77`. No active finding is in the current application POS source. This explains scope; it does not waive the repository gate. No allowlist or unrelated fixture was modified to hide findings.
2. **Governance remains blocked.** Contract decisions D-01 through D-11 lack accountable approvals. The program gate reports G1 `BLOCKED_AUTHENTIC_APPROVALS` and downstream gates blocked by dependency. Source fixes cannot manufacture those approvals or external evidence.
3. **Production receipt configuration is unproven.** The local configuration check warns that the production public receipt secret is absent. No secret was created, printed, or changed. Local `--release auto` success cannot be presented as production configuration approval.
4. **Database and browser evidence are separate.** Real PostgreSQL work and browser coverage are owned by other reviewers. Their final logs must specify scope and counts before being incorporated into release claims. Existing suites' fixture/adjacent-mock limitations remain documented in release-checks.md.

Intermediate logs that must not be called green

At resume, `frontend-tests-final.log` contained 1 failed / 95 passed across 17 suites, and `services-postgres-close.log` contained 2 failed / 5 passed. Their filenames do not establish success. Both were reported to the coordinating reviewer for resolution. The later frontend-resumed.log records 96 passing tests across 17 suites with exit 0 in frontend-resumed-exit.json. frontend-final-exits.json records the final lifecycle rerun and lint exits as 0; the frontend lint retains one `react-hooks/exhaustive-deps` ref-cleanup warning and no errors. services-postgres-exits-close.json separately records the later close rerun exit 0; consult the financial reviewer's final close log for counts and fixture limits. `services-regression.log` had 129 passing mocked tests across six suites; `services-postgres-commit.log` had three passing registry tests and explicit exit 0. These are scoped run records, not an aggregate of distinct tests.

Isolated build preparation

The requested invocation is `NEXT_DIST_DIR=.next-pos-enterprise-build-20260924` with `npm run build:app -- --skip-clean`, after receipt and POS route source stability were confirmed. It avoids the migration-deploying `npm run build` and leaves orphan-process guards enabled. The dirty `tsconfig.json` and `next-env.d.ts` were snapshotted immediately before execution; afterward remove only this build's generated include/reference if Next adds it, preserving concurrent edits and the root-owned dev include.

Execution incident, confirmed by the final wrapper summary: the npm PowerShell invocation did not forward `--skip-clean`: both the logged script header and process command line show `node scripts/build-app-safe.js` without that argument. The wrapper reported the ordinary `.next` directory as stale/incomplete because its standalone server was missing, then printed `.next cleaned: yes`; the final `.next` state was absent. It therefore unintentionally removed `E:/ohada saas/Focused projects/stoquify/.next`, which contained generated prior build output. This is a confirmed deletion of generated output, not merely an uncertain observation. The reviewer informed the coordinator when the argument issue was discovered and again when deletion was confirmed. The separate `.next-pos-audit-dev` directory and application source were preserved. No retry or further directory cleanup was performed. Any future retry must set `BUILD_APP_SKIP_CLEAN=1` explicitly and verify its effective arguments rather than relying on npm forwarding.

An existing wrapper limitation is explicit: `inspectNextOutput()` and final output validation inspect `.next` even when `NEXT_DIST_DIR` changes Next's actual output. Therefore record both the Next child exit and wrapper exit and inspect the isolated output directly. A `failed-post-build-output` wrapper result cannot be silently relabeled as a fully passing `build:app`. No wrapper change is included in this review.

The reviewed `next.config.mjs` and wrapper contain no TypeScript build-error bypass or lint bypass. The build compiled successfully and entered lint/type validation, printed six unrelated lint warnings, then reached its timeout before validation completion or packaging was established. It does not constitute a passing production build.

The cash-history location fix was applied during compilation at 03:34:45 UTC. cash-history-location-final.json and the owner's baseline preserve its before/after source provenance. The generated cash-history page contained the payment reader but did not contain either new purchase-order location predicate marker; see compiled-cash-history.json. This artifact is not established to contain the final correction. The independent final project typecheck was started after the fix and binds the final source hash; it is reported separately from build output. No automatic build retry was performed.

After both build and standalone typecheck stopped, the build-added tsconfig include and generated `next-env.d.ts` reference were removed. The restoration helper first refused because Next also reordered the include list; after verifying that ordering was the only additional difference, it restored the exact pre-build bytes. build-config-cleanup.json proves both hashes matched the saved pre-build hashes and that the coordinating reviewer's `.next-pos-audit-dev` include was preserved for their cleanup. The coordinator then removed only that dev include; dev-config-cleanup.json records final tsconfig SHA-256 `254b180507594bf66fe34990b43ff01937db12b09f9ea30316eba451cedda328` with no other configuration field or directory changed. These configuration cleanups did not restore the generated `.next` output removed earlier.

The failed standalone typecheck began at 03:36:44 UTC. The guarded current hook (`"code" in response ? response.code : undefined`) was modified at 03:43:59 UTC during that run, with SHA-256 `F03A12CBCCF5CF28277C8B088645FC1033352B883E7AA7AAE6DB8763A61DB0B7`; the author of that concurrent correction was not established by this reviewer. The failed log is retained. After configuration cleanup and a source-stable handoff, a separate final check ran from 03:56:06 to 04:12:21 UTC and **passed with exit 0 and no diagnostics**. All seven source/config hashes matched at start and completion; the comparison also exited 0. This establishes current-source TypeScript validation separately from the unsuccessful and potentially stale production build.

Handoff

The coordinating reviewer owns application changes, browser checks, final PostgreSQL interpretation, and the consolidated findings/release decision. No live sale, refund, void, charge, receipt delivery, production deploy, or

migration was performed by this reviewer. Preserve the recent full-height checkout and retained-request behavior while completing those checks.